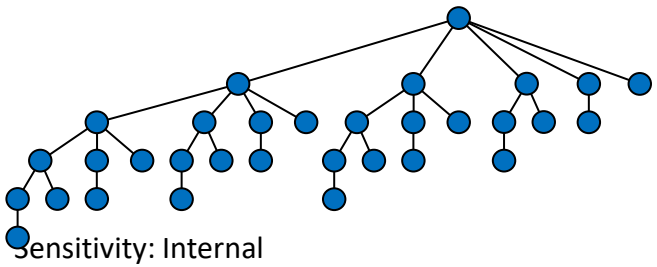
Lecture 20. Shortest Paths

CpSc 212: Algorithms and Data Structures

Brian C. Dean



School of Computing
Clemson University

Fall, 2012

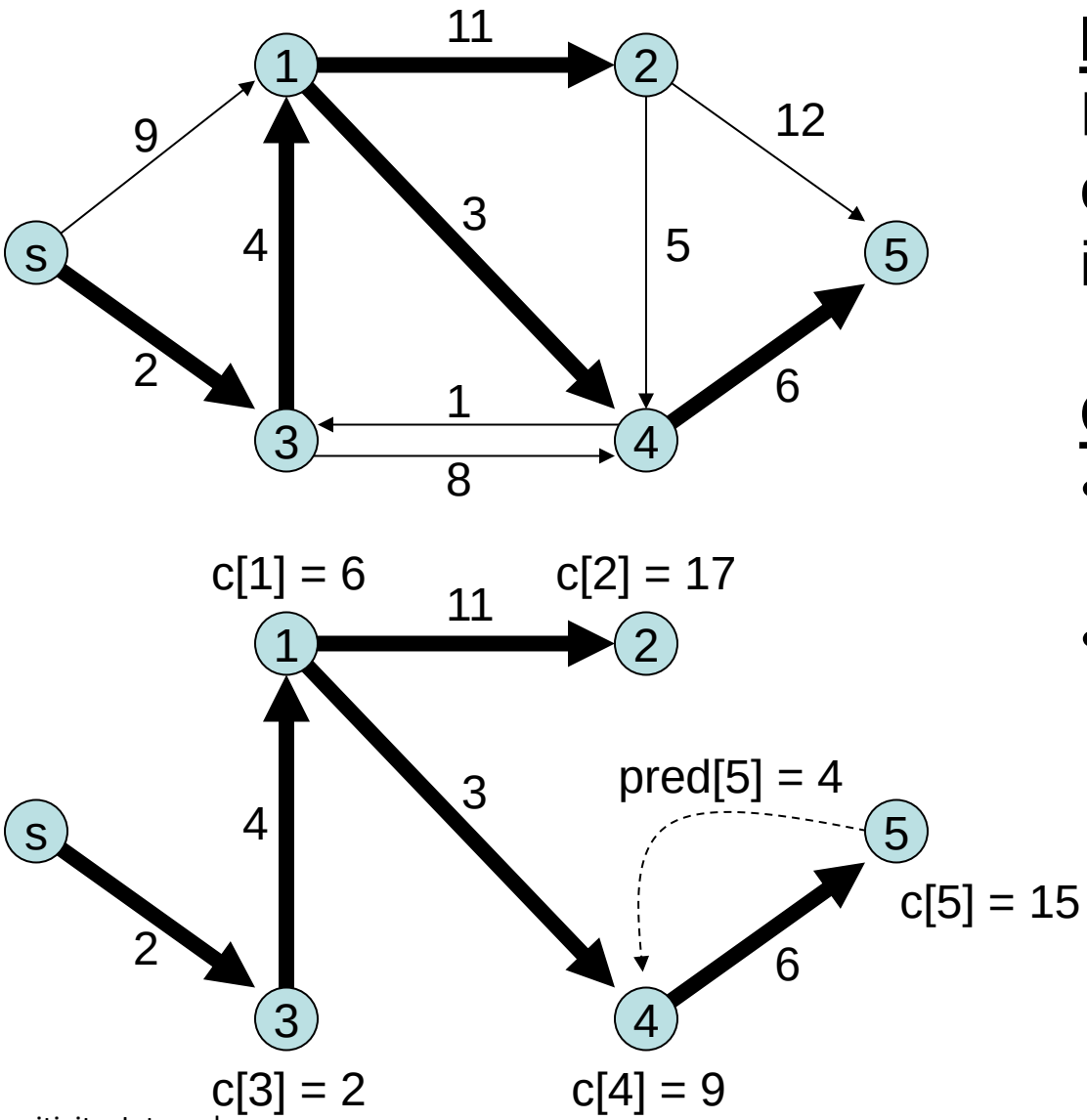
Sensitivity: Internal

Types of Shortest Path Problems

- **Single-source single-destination:** find the shortest path from s to t in a (directed) graph.
- **Single-source:** find the shortest path from s to every other node in a (directed) graph.
- **All-pairs:** find the shortest path from every node to every other node.

Sensitivity: Internal

Single-Source Shortest Paths



Input:

Directed graph with costs/weights/lengths on its edges.

Output:

- c[j]: cost of shortest $s \rightarrow j$ path
- pred[j]: previous node (before j) on a shortest $s \rightarrow j$ path.

Sensitivity: Internal

Single-Source Shortest Paths: Algorithms

- Easy cases that we can solve in linear time:
  - Unweighted graphs: use **breadth-first search**.
  - Directed acyclic graphs (DAGs): use **dynamic programming**.
- If edge costs are nonnegative, we'll use **Dijkstra's algorithm**.
- If some edge costs are negative, we use the **Bellman-Ford algorithm**.

Sensitivity: Internal

Breadth-First Search

- Depth-first search dives as deeply as possible into a graph until it can go no further, then it backs up and tries to branch.
- In contrast, a **breadth-first search** (BFS) starting at some source node s will visit s, then all nodes 1 hop away from s, then all nodes 2 hops away from s, etc.
- Like DFS, we can also use BFS to find connected components or answer “find a path from i to j” queries.
  - BFS finds a path from i to j having fewest edges.

Breadth-First Search

```
BFS(s):
   For all nodes i:
      pred[i] = null
      dist[i] = Infinity
   Q = {s}
   dist[s] = 0
   While Q is nonempty:
        i = next node in Q
        For all nodes j such that (i,j) is an edge:
           If dist[j] = Infinity,
              pred[j] = i
              dist[j] = dist[i] + 1
              Append j to the end of Q
```

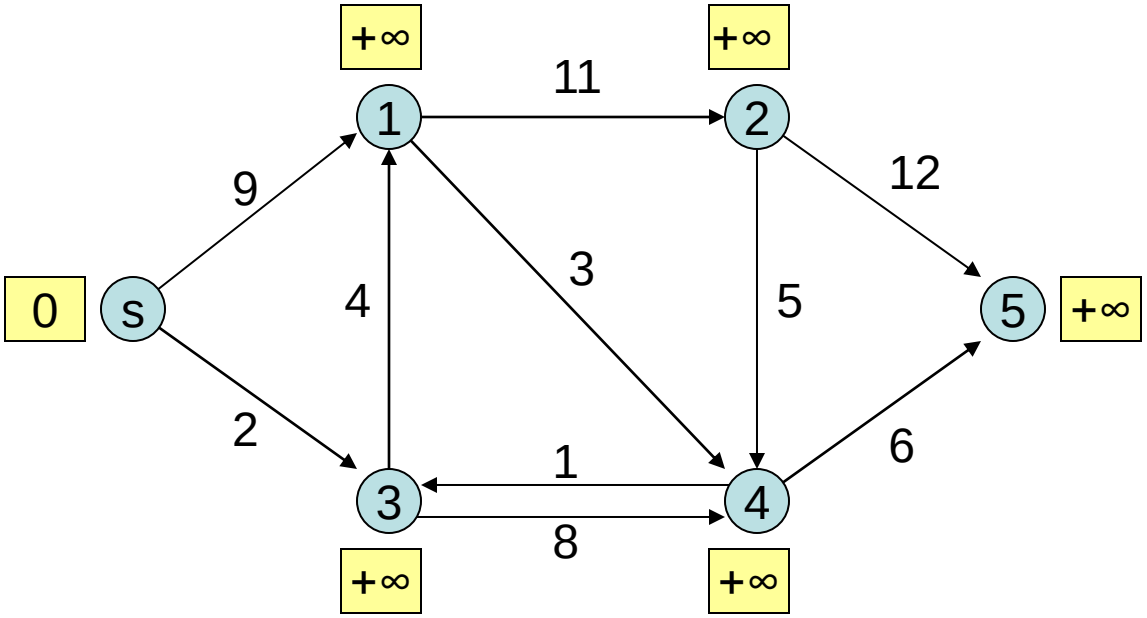
- If Q is implemented as a stack rather than as a FIFO queue, we get DFS rather than BFS!

Negative-Cost Cycles

- If negative-cost edges exist:
  - We need to use the O(mn) Bellman-Ford algorithm.
  - We can only find shortest paths if there are no negative-cost directed cycles (Bellman-Ford can at least detect whether negative cycles exist).
  - If our graph is undirected, we typically assume it has no negative-cost edges to begin with.
- Longest path problems convert to shortest path problems by negating edge lengths, so positive-cost cycles are bad for these problems
  - So longest path problems are rarely tractable, since most graphs have positive-cost cycles.
  - Except DAGs, that is…

Sensitivity: Internal

Updating Cost Labels

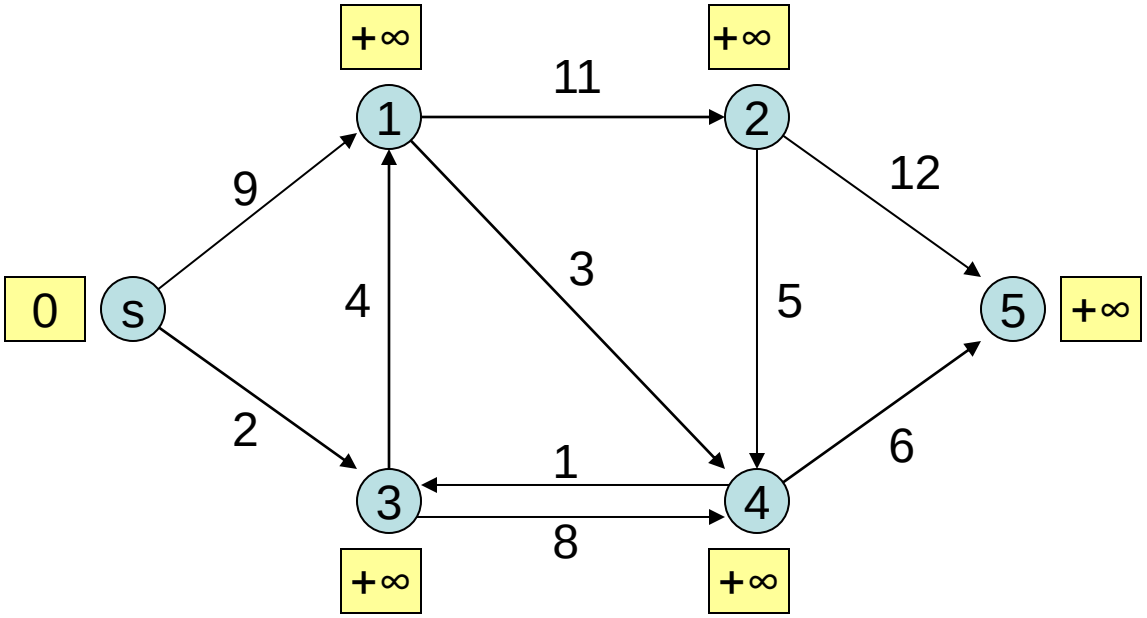


Initially, all nodes have cost labels set to +∞, except the source, which has cost label 0.

We'd like to improve the cost labels of all nodes so they reflect the costs of shortest paths.

Sensitivity: Internal

Dijkstra's Algorithm – Example

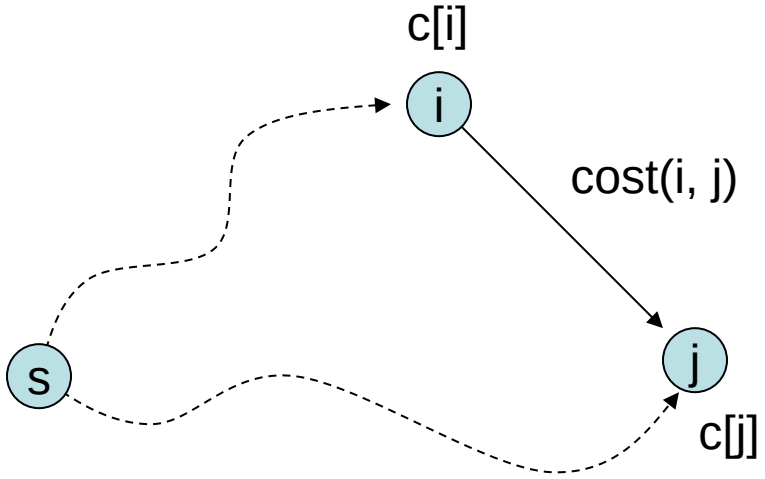


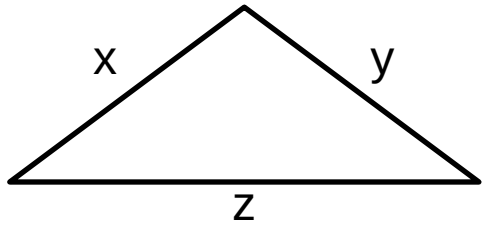
Initialization.

Improving Cost labels

- Shortest path costs satisfy the triangle inequality:

  c[i] + cost(i, j) ≥ c[j] for every edge (i, j).





x + y ≥ z

- If this is violated for some edge (i, j), we can improve the cost label of node j:

```
Tighten(i, j):  (also called Relax(i, j))
   If c[i] + cost(i, j) < c[j],
      c[j] = c[i] + cost(i, j)
      pred[j] = i
```

Sensitivity: Internal

The Bellman-Ford Algorithm

```
Bellman-Ford:
   c[s] = 0, c[j ≠ s] = +∞
   Repeat n-1 times:
      Tighten every edge in the graph
   If c[i] + cost(i, j) < c[j] for any edge (i, j)
      Output "negative cycle detected!"
```

- O(mn) time.
- Computes single-source shortest paths in the presence of negative edge costs, and can detect negative cycles.
- Analyze using induction: after k iterations of the main loop, each label c[j] reflects the cost of the shortest k-hop path from s to j.

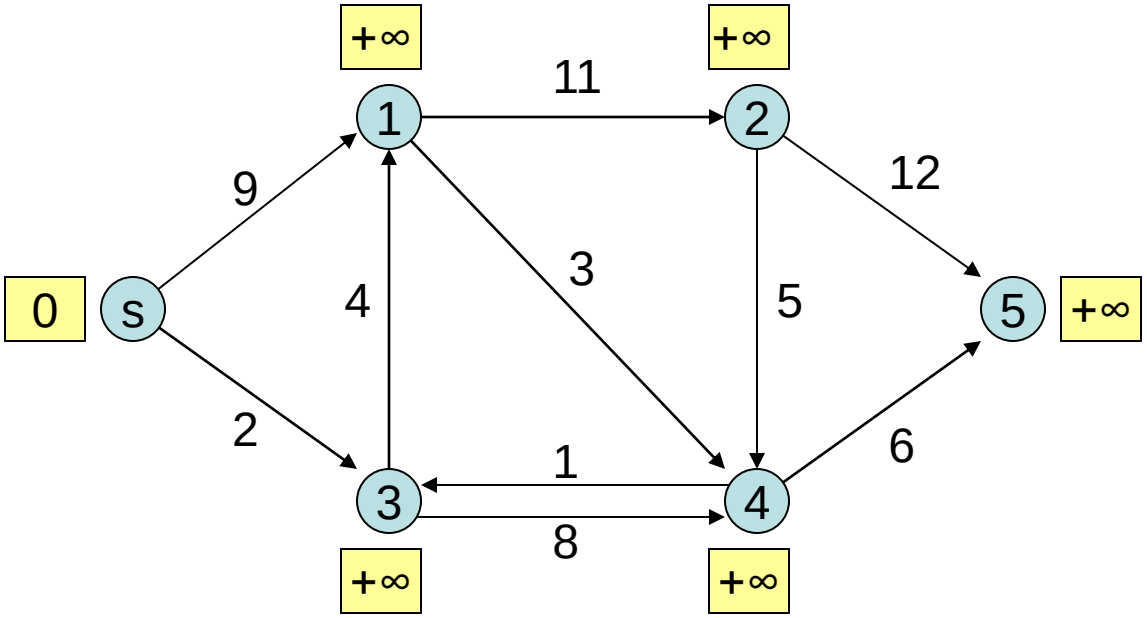
Dijkstra's Algorithm

```
Dijkstra:
   c[s] = 0, c[j ≠ s] = +∞
   Build a priority queue Q on the c[j]'s
   While Q is nonempty:
      Remove from Q the node i having minimum label c[i]
           (at this point we know c[i] is correct)
      Tighten all edges emanating from i
```

- Only works if edge costs are nonnegative.
- Running time depends on priority queue:
  - n inserts (during initialization)
  - n remove-mins
  - at most m decrease-keys
- Using a binary heap, all these operations run in O(log n) time, so total is O(m log n).

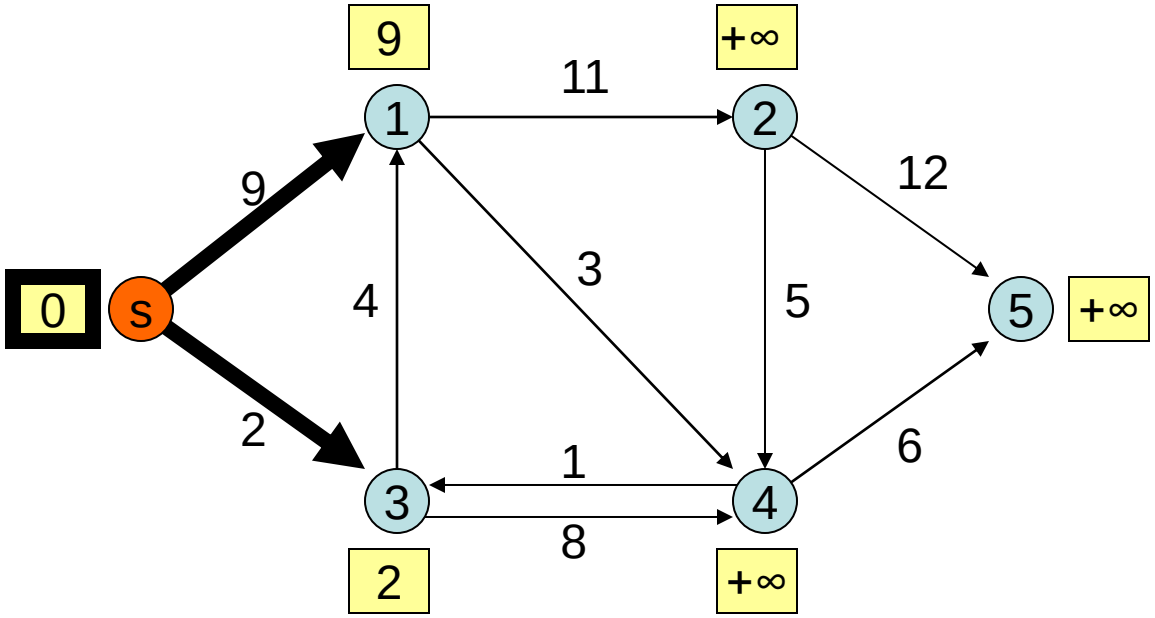
Sensitivity: Internal

Dijkstra's Algorithm – Example



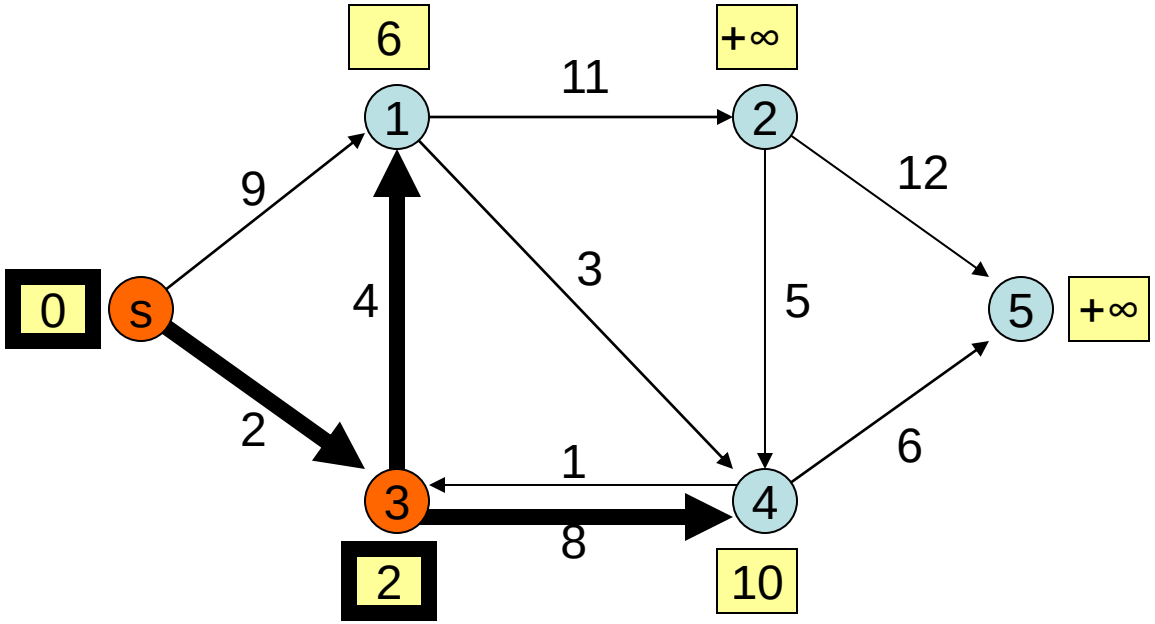
Initialization.

Dijkstra's Algorithm – Example



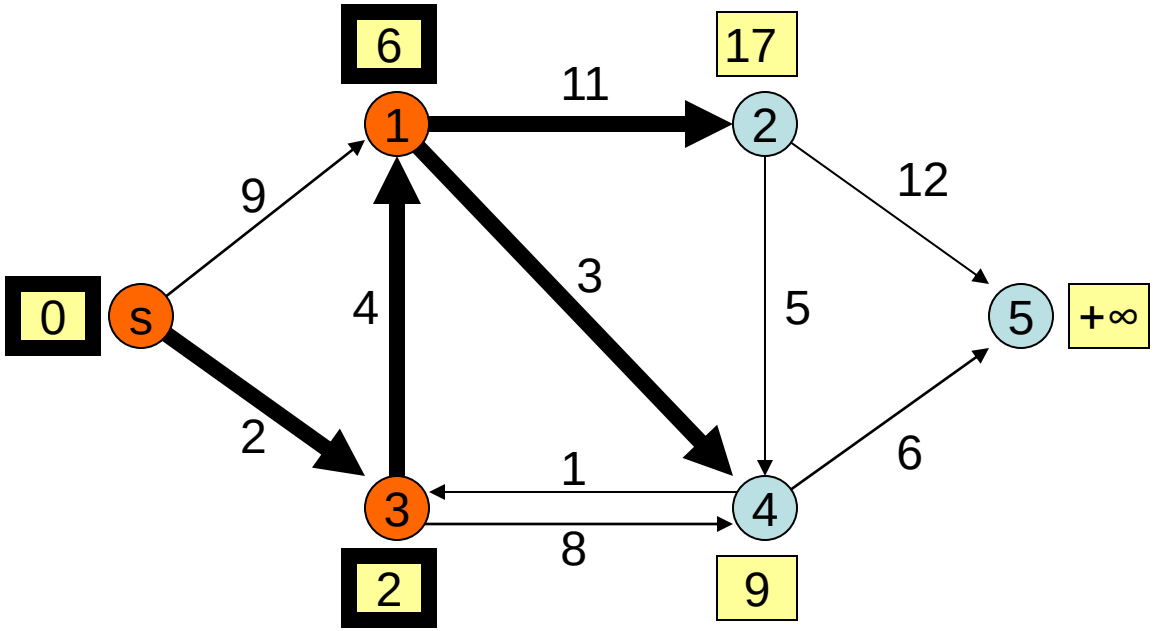
Examine the source node s, so s becomes "permanently" labeled.
Tighten all edges leaving s.

Dijkstra's Algorithm – Example



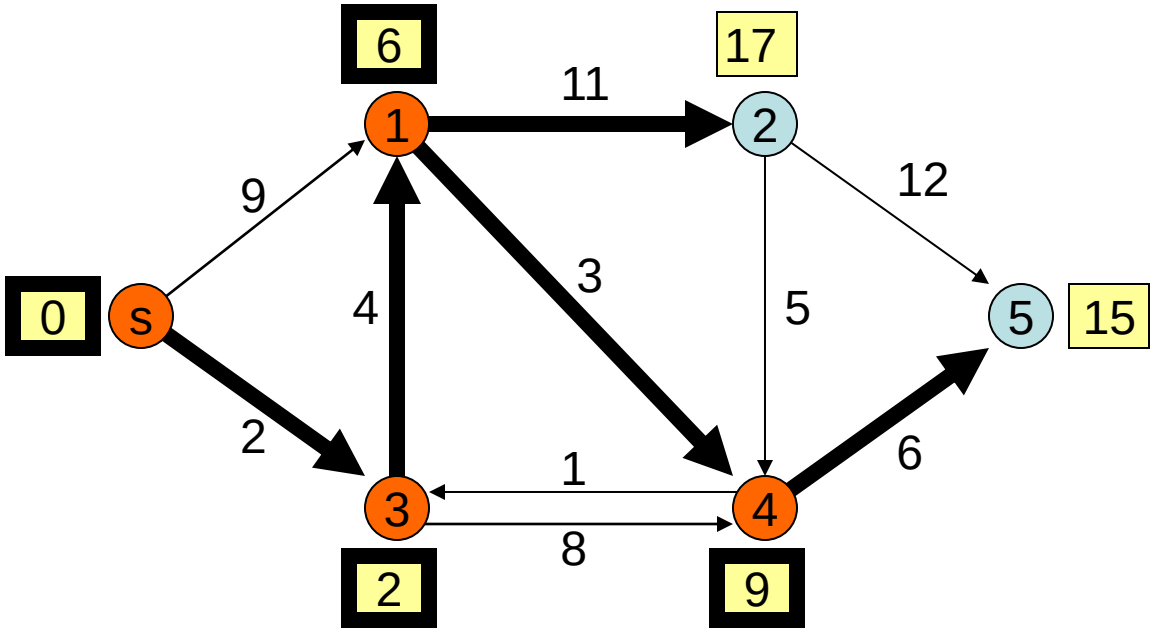
Examine node 3, which becomes permanently labeled.
Tighten all edges leaving node 3.

Dijkstra's Algorithm – Example



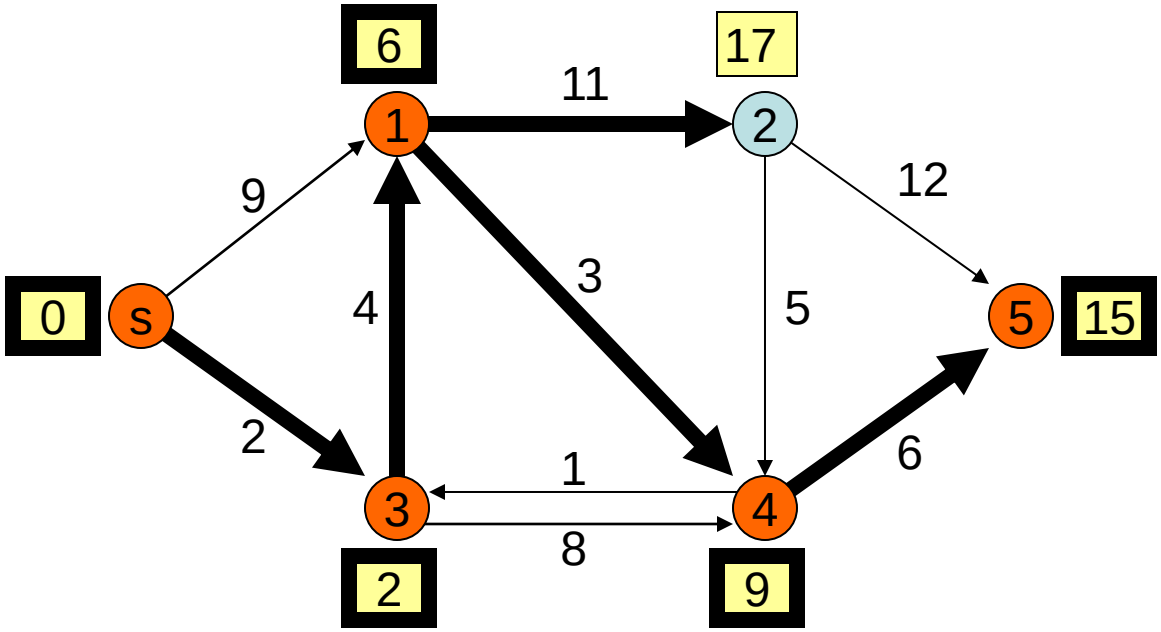
Examine node 1, which becomes permanently labeled.
Tighten all edges leaving node 1.

Dijkstra's Algorithm – Example



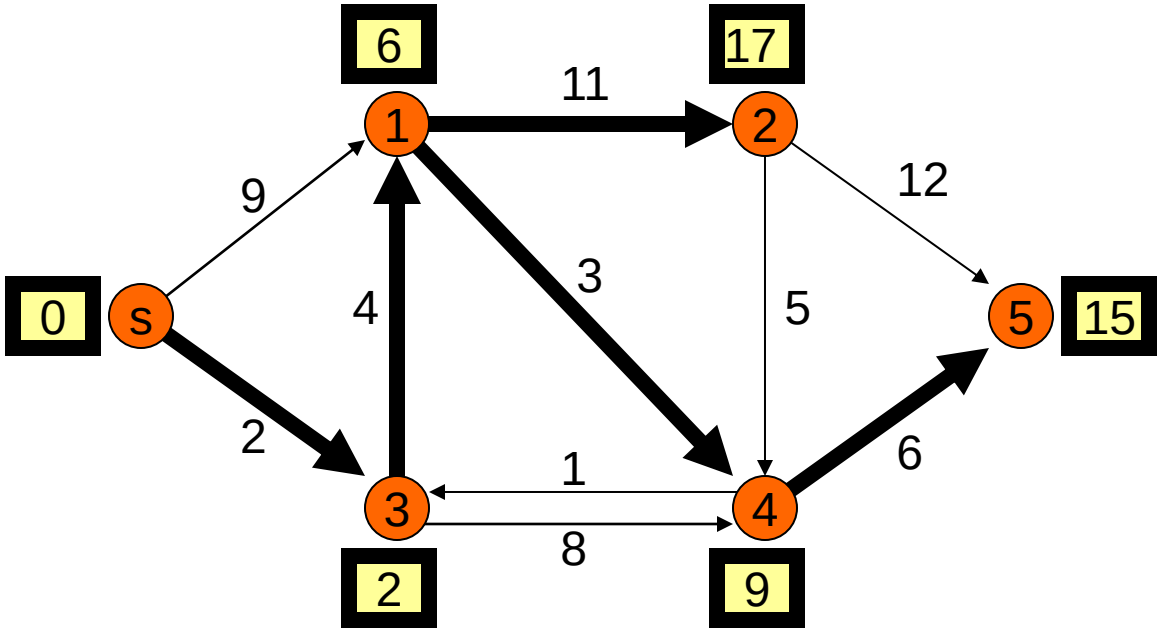
Examine node 4, which becomes permanently labeled.
Tighten all edges leaving node 4.

Dijkstra's Algorithm – Example



Examine node 5, which becomes permanently labeled.
Tighten all edges leaving node 5 (there aren't any).

Dijkstra's Algorithm – Example



Examine node 2, which becomes permanently labeled.
Algorithm terminates!

Dijkstra's Algorithm – Correctness

- Just like BFS, Dijkstra visits all nodes in increasing order of distance from the source, and visits each node exactly once.
- Think of Dijkstra as maintaining two sets:

  S: visited nodes that are "permanently" labeled.

  T: remaining nodes that are temporarily labeled (whose cost labels are still upper bounds)
- Every iteration takes the node from T with minimum label and adds it to S.
- **Claim**: when we visit node i, c[i] will be the correct shortest path cost from s to i.

Sensitivity: Internal

Dijkstra's Algorithm – Running Time

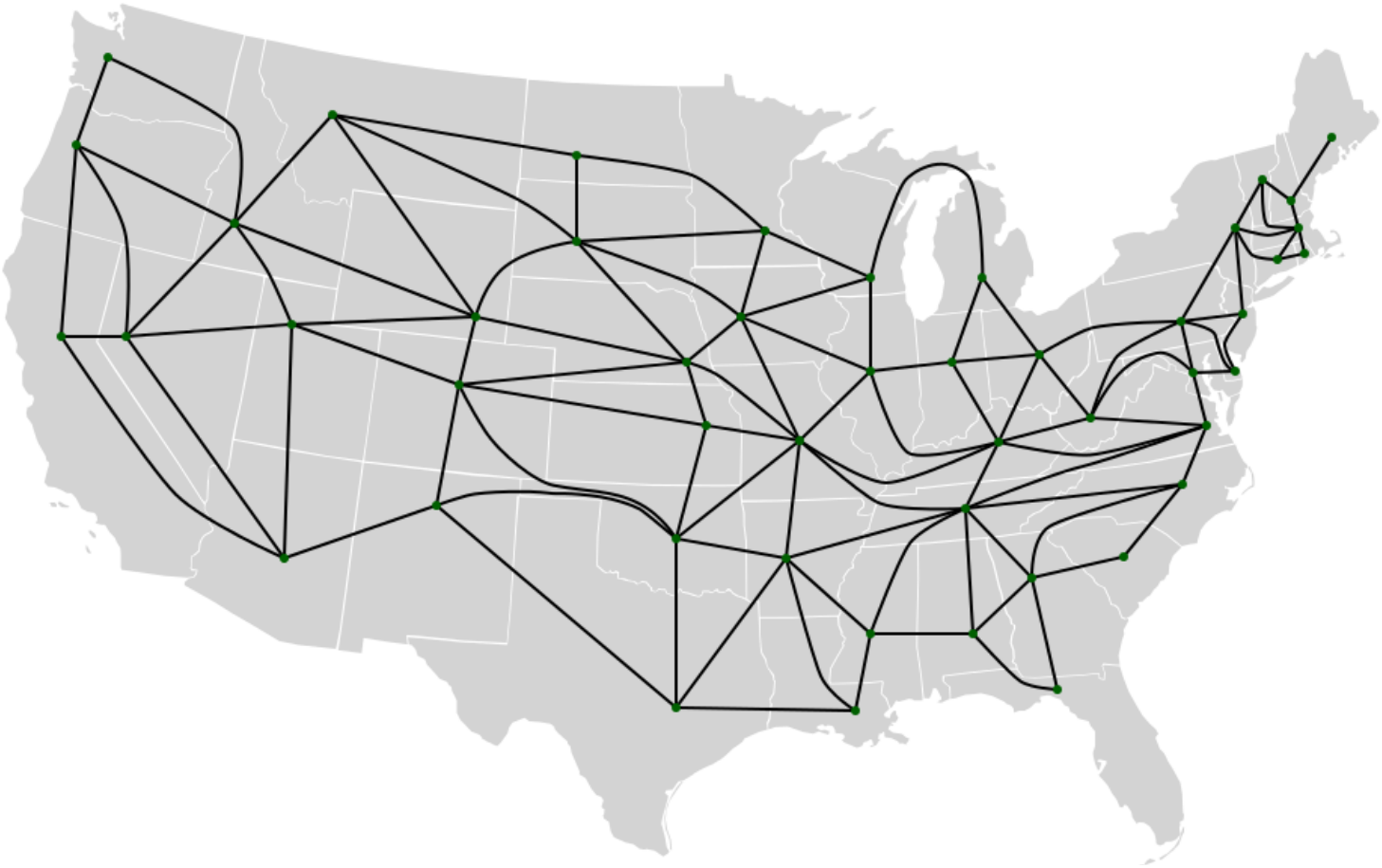
	Remove-Min O(n) times	**Decrease Key O(m) times**	**Total Runtime**
Unsorted Array	O(n)	O(1)	$O(n^2)$
Binary Heap	O(log n)	O(log n)	O(m log n)
Fibonacci Heap	O(log n)	O(1) amortized	O(m + n log n)

- The unsorted array is better for a dense graph, and the binary heap is better for a sparse graph.
- The Fibonacci heap bound is always at least as good as both other bounds in any graph!
  - In practice, however, Fibonacci heaps are so complicated that it's hard to find instances for which they run faster…

Sensitivity: Internal

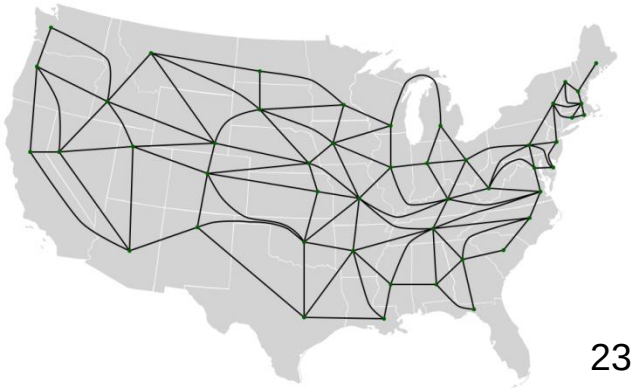
Recall: Centrality

- What nodes are most “central” in a graph?



Centrality & Shortest Paths

- Many notions of centrality are based on shortest paths:
  - Eccentricity(x) = max dist(x,y) over other nodes y; a graph “center” is a node with minimum eccentricity.
  - “Betweenness” centrality of an edge or node: fraction of all point-to-point shortest paths passing through that edge/node.
- Other prominent notions of centrality include:
  - Degree centrality: high degree treated as “more central”
  - Eigenvector centrality (centrality proportional to probability a node is visited by a random walk in steady state); used by Google PageRank.



Sensitivity: Internal

Formulating Shortest Path Problems: Change the Problem, not the Algorithm…

- In an automobile transportation network, suppose right turns can be made instantly, while left turns take 1 extra unit of time…

Sensitivity: Internal

Formulating Shortest Path Problems: Change the Problem, not the Algorithm…

- In an automobile transportation network, suppose right turns can be made instantly, while left turns take 1 extra unit of time…
- Suppose we have N jobs to complete. Each job has a duration and possibly a set of prerequisite jobs. Working in parallel, how quickly can we finish all the jobs?

Sensitivity: Internal

Formulating Shortest Path Problems: Change the Problem, not the Algorithm…

- In an automobile transportation network, suppose right turns can be made instantly, while left turns take 1 extra unit of time…
- Suppose we have N jobs to complete. Each job has a duration and possibly a set of prerequisite jobs. Working in parallel, how quickly can we finish all the jobs?
- Suppose some roads charge a $1 toll. How do we find a shortest path if we can pay at most $3 worth of tolls?

Sensitivity: Internal

Formulating Shortest Path Problems: Change the Problem, not the Algorithm…

- In an automobile transportation network, suppose right turns can be made instantly, while left turns take 1 extra unit of time…
- Suppose we have N jobs to complete. Each job has a duration and possibly a set of prerequisite jobs. Working in parallel, how quickly can we finish all the jobs?
- Suppose some roads charge a $1 toll. How do we find a shortest path if we can pay at most $3 worth of tolls?
- What is the second-shortest s-t path?

Sensitivity: Internal

Postage

- We have N different denominations of stamps (e.g., 1 cent, 23 cent, 47 cent).
- We can use as many stamps of each denomination as we wish.
- What is the minimum number of stamps we need in order to construct exactly C cents worth of postage?

Sensitivity: Internal

“Knapsack” Problems

- We have N items, with values $v_1 \ldots v_n$ and integer sizes $s_1 \ldots s_n$.
- What is the maximum value we can pack into a knapsack of capacity C?

  … if we can include multiple copies of the same item?

  … if we can only include at most 1 copy of each item?

Sensitivity: Internal